Machine Learning in Python

Neural Networks I: Foundations and Optimization

Cristian A. Marocico, A. Emin Tatar

Center for Information Technology
University of Groningen

Tuesday, February 17th 2026

Outline

What is an Artificial Neural Network?

Definition

Definition

Key Properties

Example

What is an Artificial Neural Network?

Definition

Definition

An Artificial Neural Network (ANN) is a computational model composed of layers of interconnected nodes ("neurons") that process information by transmitting signals to one another.

Key Properties

Example

What is an Artificial Neural Network?

Definition

Definition

An Artificial Neural Network (ANN) is a computational model composed of layers of interconnected nodes ("neurons") that process information by transmitting signals to one another.

Key Properties

Example

- Foundation of Deep Learning

What is an Artificial Neural Network?

Definition

Definition

An Artificial Neural Network (ANN) is a computational model composed of layers of interconnected nodes ("neurons") that process information by transmitting signals to one another.

Key Properties

Example

- Foundation of Deep Learning
- Can learn complex, non-linear relationships

What is an Artificial Neural Network?

Definition

Definition

An Artificial Neural Network (ANN) is a computational model composed of layers of interconnected nodes ("neurons") that process information by transmitting signals to one another.

Key Properties

Example

- Foundation of Deep Learning
- Can learn complex, non-linear relationships
- Powers image recognition, voice assistants, medical diagnosis, and more

Key Components of a Neural Network

The Building Blocks

Key Components of a Neural Network

The Building Blocks

1. Neurons (Nodes): Basic processing units that receive inputs, perform a calculation, and produce an output

Key Components of a Neural Network

The Building Blocks

1. Neurons (Nodes): Basic processing units that receive inputs, perform a calculation, and produce an output
2. Weights: Connections between neurons that determine influence; learning = adjusting weights

Key Components of a Neural Network

The Building Blocks

1. Neurons (Nodes): Basic processing units that receive inputs, perform a calculation, and produce an output
2. Weights: Connections between neurons that determine influence; learning = adjusting weights
3. Layers:

Key Components of a Neural Network

The Building Blocks

1. Neurons (Nodes): Basic processing units that receive inputs, perform a calculation, and produce an output
2. Weights: Connections between neurons that determine influence; learning = adjusting weights
3. Layers:
   - Input Layer: Receives raw data

Key Components of a Neural Network

The Building Blocks

1. Neurons (Nodes): Basic processing units that receive inputs, perform a calculation, and produce an output
2. Weights: Connections between neurons that determine influence; learning = adjusting weights
3. Layers:
   - Input Layer: Receives raw data
   - Hidden Layers: Feature extraction (the "magic")

Key Components of a Neural Network

The Building Blocks

1. Neurons (Nodes): Basic processing units that receive inputs, perform a calculation, and produce an output
2. Weights: Connections between neurons that determine influence; learning = adjusting weights
3. Layers:
   - Input Layer: Receives raw data
   - Hidden Layers: Feature extraction (the "magic")
   - Output Layer: Final prediction

Key Components of a Neural Network

The Building Blocks

1. Neurons (Nodes): Basic processing units that receive inputs, perform a calculation, and produce an output
2. Weights: Connections between neurons that determine influence; learning = adjusting weights
3. Layers:
   - Input Layer: Receives raw data
   - Hidden Layers: Feature extraction (the "magic")
   - Output Layer: Final prediction
4. Activation Function: Introduces non-linearity, allowing the network to learn curves

Biological Inspiration

From Biology to Mathematics

Biological Inspiration

From Biology to Mathematics

ANNs are loosely inspired by biological neural networks in the brain:

Biological Inspiration

From Biology to Mathematics

ANNs are loosely inspired by biological neural networks in the brain:

Biological Neuron	Artificial Neuron	Role
Dendrites	Inputs (x)	Receives signals
Synapse	Weights (w)	Controls signal strength
Cell Body	Summation ($\sum$)	Aggregates signals
Axon	Activation	Transmits output

The Perceptron (1958)

The Simplest Neural Network

Definition

Mathematical Formulation

1. Weighted Sum: $z = w_1 x_1 + w_2 x_2 + \cdots + b$
2. Activation:

$$\text{Output} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

The Perceptron (1958)

The Simplest Neural Network

Definition

The Perceptron is a single-layer network that takes inputs, weighs them, and makes a binary decision.

Mathematical Formulation

1. Weighted Sum: $z = w_1 x_1 + w_2 x_2 + \cdots + b$
2. Activation:

$$\text{Output} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

The Perceptron (1958)

The Simplest Neural Network

Definition

The Perceptron is a single-layer network that takes inputs, weighs them, and makes a binary decision.

Mathematical Formulation

1. Weighted Sum: $z = w_1 x_1 + w_2 x_2 + \cdots + b$
2. Activation:

$$\text{Output} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

The Perceptron (1958)

The Simplest Neural Network

Definition

The Perceptron is a single-layer network that takes inputs, weighs them, and makes a binary decision.

Mathematical Formulation

1. Weighted Sum: $z = w_1 x_1 + w_2 x_2 + \cdots + b$
2. Activation:

$$\text{Output} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

This can solve linearly separable problems (drawing a straight line between classes).

The XOR Problem

The Fatal Flaw of Single Perceptrons

The Limitation

Example

The XOR Problem

The Fatal Flaw of Single Perceptrons

x_1	x_2	AND	XOR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

The Limitation

Example

The XOR Problem

The Fatal Flaw of Single Perceptrons

x_1	x_2	AND	XOR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

The Limitation

Example

- AND Gate: Linearly separable (one line can separate classes)

The XOR Problem

The Fatal Flaw of Single Perceptrons

x_1	x_2	AND	XOR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

The Limitation

Example

- AND Gate: Linearly separable (one line can separate classes)
- XOR Gate: NOT linearly separable (needs a curve or multiple lines)

The XOR Problem

The Fatal Flaw of Single Perceptrons

x_1	x_2	AND	XOR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

The Limitation

Example

- AND Gate: Linearly separable (one line can separate classes)
- XOR Gate: NOT linearly separable (needs a curve or multiple lines)
- This limitation froze AI research until Multi-Layer Perceptrons (MLPs) emerged

From One to Many

Stacking Neurons

Definition

Architecture

From One to Many

Stacking Neurons

Definition

A Multilayer Perceptron (MLP) or Feedforward Neural Network connects many neurons in layers to solve complex, non-linear problems.

Architecture

From One to Many

Stacking Neurons

Definition

A Multilayer Perceptron (MLP) or Feedforward Neural Network connects many neurons in layers to solve complex, non-linear problems.

Architecture

1. Input Layer: Receives raw feature vector X (no computation)

From One to Many

Stacking Neurons

Definition

A Multilayer Perceptron (MLP) or Feedforward Neural Network connects many neurons in layers to solve complex, non-linear problems.

Architecture

1. Input Layer: Receives raw feature vector X (no computation)
2. Hidden Layers: Perform computation and extract features; more layers = "deeper" network

From One to Many

Stacking Neurons

Definition

A Multilayer Perceptron (MLP) or Feedforward Neural Network connects many neurons in layers to solve complex, non-linear problems.

Architecture

1. Input Layer: Receives raw feature vector X (no computation)
2. Hidden Layers: Perform computation and extract features; more layers = "deeper" network
3. Output Layer: Produces final prediction (e.g., probability of class)

Activation Functions

Why Activation Functions?

Common Activation Functions

Example

Function	Formula	Range	Usage
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$	Output (Binary Class.)
Tanh	$\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$(-1, 1)$	Hidden layers (Legacy)
ReLU	$\max(0, z)$	$[0, \infty)$	Standard for Hidden
Softmax	$\frac{e^{z_i}}{\sum e^{z_j}}$	$(0, 1)$	Output (Multiclass)

Activation Functions

Why Activation Functions?

Without activation functions, an MLP is just a giant Linear Regression (linear + linear = linear). Activation functions introduce non-linearity, allowing networks to learn curves.

Common Activation Functions

Example

Function	Formula	Range	Usage
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$	Output (Binary Class.)
Tanh	$\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$(-1, 1)$	Hidden layers (Legacy)
ReLU	$\max(0, z)$	$[0, \infty)$	Standard for Hidden
Softmax	$\frac{e^{z_i}}{\sum e^{z_j}}$	$(0, 1)$	Output (Multiclass)

Activation Functions

Why Activation Functions?

Without activation functions, an MLP is just a giant Linear Regression (linear + linear = linear). Activation functions introduce non-linearity, allowing networks to learn curves.

Common Activation Functions

Example

Function	Formula	Range	Usage
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$	Output (Binary Class.)
Tanh	$\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$(-1, 1)$	Hidden layers (Legacy)
ReLU	$\max(0, z)$	$[0, \infty)$	Standard for Hidden
Softmax	$\frac{e^{z_i}}{\sum e^{z_j}}$	$(0, 1)$	Output (Multiclass)

Forward Propagation

How Data Flows Through the Network

Forward Propagation

How Data Flows Through the Network

For a single layer l:

Forward Propagation

How Data Flows Through the Network

For a single layer l:

1. Linear Step (Z): Weighted sum of inputs from previous layer

$$Z^{[l]} = W^{[l]} \cdot A^{[l-1]} + b^{[l]}$$

Forward Propagation

How Data Flows Through the Network

For a single layer l:

1. Linear Step (Z): Weighted sum of inputs from previous layer

$$Z^{[l]} = W^{[l]} \cdot A^{[l-1]} + b^{[l]}$$

2. Activation Step (A): Apply activation function

$$A^{[l]} = g(Z^{[l]})$$

Forward Propagation

How Data Flows Through the Network

For a single layer l:

1. Linear Step (Z): Weighted sum of inputs from previous layer

$$Z^{[l]} = W^{[l]} \cdot A^{[l-1]} + b^{[l]}$$

2. Activation Step (A): Apply activation function

$$A^{[l]} = g(Z^{[l]})$$

Forward Propagation

How Data Flows Through the Network

For a single layer l:

1. Linear Step (Z): Weighted sum of inputs from previous layer

$$Z^{[l]} = W^{[l]} \cdot A^{[l-1]} + b^{[l]}$$

2. Activation Step (A): Apply activation function

$$A^{[l]} = g(Z^{[l]})$$

This process repeats until the output layer. The magic: `np.dot` handles all connections automatically.

Network Capacity: Width vs Depth

Architectural Choices

Trade-offs

Example

Network Capacity: Width vs Depth

Architectural Choices

Architecture	Structure	Characteristics
Wide	1 layer, 100 neurons	Simple patterns, fast
Deep	5 layers, 10 neurons	Hierarchical features, complex

Trade-offs

Example

Network Capacity: Width vs Depth

Architectural Choices

Architecture	Structure	Characteristics
Wide	1 layer, 100 neurons	Simple patterns, fast
Deep	5 layers, 10 neurons	Hierarchical features, complex

Trade-offs

Example

- Deeper networks learn hierarchical representations (edges $\rightarrow$ parts $\rightarrow$ objects)

Network Capacity: Width vs Depth

Architectural Choices

Architecture	Structure	Characteristics
Wide	1 layer, 100 neurons	Simple patterns, fast
Deep	5 layers, 10 neurons	Hierarchical features, complex

Trade-offs

Example

- Deeper networks learn hierarchical representations (edges $\rightarrow$ parts $\rightarrow$ objects)
- Wider networks can approximate functions but may need more parameters

Network Capacity: Width vs Depth

Architectural Choices

Architecture	Structure	Characteristics
Wide	1 layer, 100 neurons	Simple patterns, fast
Deep	5 layers, 10 neurons	Hierarchical features, complex

Trade-offs

Example

- Deeper networks learn hierarchical representations (edges $\rightarrow$ parts $\rightarrow$ objects)
- Wider networks can approximate functions but may need more parameters
- In practice: start simple, add complexity if needed

The Goal of Training

From Random to Learned

Three Components

Example

The Goal of Training

From Random to Learned

A neural network starts with random weights. It knows nothing. Training adjusts these weights to minimize prediction error.

Three Components

Example

The Goal of Training

From Random to Learned

A neural network starts with random weights. It knows nothing. Training adjusts these weights to minimize prediction error.

Three Components

Example

1. Loss Function: How we measure error

The Goal of Training

From Random to Learned

A neural network starts with random weights. It knows nothing. Training adjusts these weights to minimize prediction error.

Three Components

Example

1. Loss Function: How we measure error
2. Optimizer: How we update weights

The Goal of Training

From Random to Learned

A neural network starts with random weights. It knows nothing. Training adjusts these weights to minimize prediction error.

Three Components

Example

1. Loss Function: How we measure error
2. Optimizer: How we update weights
3. Backpropagation: How we calculate the updates

Loss Functions

The Scorecard

Definition

Common Loss Functions

Problem Type	Loss Function	Formula
Regression	Mean Squared Error	$\frac{1}{n}\sum(y-\hat{y})^2$
Classification	Cross-Entropy	$-\sum y \log(\hat{y})$

Loss Functions

The Scorecard

Definition

The Loss Function (Cost Function) calculates a single number representing "how bad" the model is. Goal: drive it to zero.

Common Loss Functions

Problem Type	Loss Function	Formula
Regression	Mean Squared Error	$\frac{1}{n}\sum(y-\hat{y})^2$
Classification	Cross-Entropy	$-\sum y \log(\hat{y})$

Loss Functions

The Scorecard

Definition

The Loss Function (Cost Function) calculates a single number representing "how bad" the model is. Goal: drive it to zero.

Common Loss Functions

Problem Type	Loss Function	Formula
Regression	Mean Squared Error	$\frac{1}{n}\sum(y-\hat{y})^2$
Classification	Cross-Entropy	$-\sum y \log(\hat{y})$

Loss Functions

The Scorecard

Definition

The Loss Function (Cost Function) calculates a single number representing "how bad" the model is. Goal: drive it to zero.

Common Loss Functions

Problem Type	Loss Function	Formula
Regression	Mean Squared Error	$\frac{1}{n}\sum(y-\hat{y})^2$
Classification	Cross-Entropy	$-\sum y \log(\hat{y})$

- MSE: Penalizes large errors heavily (prices, temperatures)
- Cross-Entropy: Penalizes confident wrong answers (probabilities)

Stochastic Gradient Descent (SGD)

Optimization Strategy

Stochastic Gradient Descent

Definition

SGD updates weights after seeing just one example (or a small "mini-batch").

Stochastic Gradient Descent (SGD)

Optimization Strategy

Gradient Descent minimizes loss by following the slope downhill. Batch GD calculates gradient for entire dataset — slow and memory-intensive.

Stochastic Gradient Descent

Definition

SGD updates weights after seeing just one example (or a small "mini-batch").

Stochastic Gradient Descent (SGD)

Optimization Strategy

Gradient Descent minimizes loss by following the slope downhill. Batch GD calculates gradient for entire dataset — slow and memory-intensive.

Stochastic Gradient Descent

Definition

SGD updates weights after seeing just one example (or a small "mini-batch").

Stochastic Gradient Descent (SGD)

Optimization Strategy

Gradient Descent minimizes loss by following the slope downhill. Batch GD calculates gradient for entire dataset — slow and memory-intensive.

Stochastic Gradient Descent

Definition

SGD updates weights after seeing just one example (or a small "mini-batch").

- Pros: Faster, noise helps escape local minima

Stochastic Gradient Descent (SGD)

Optimization Strategy

Gradient Descent minimizes loss by following the slope downhill. Batch GD calculates gradient for entire dataset — slow and memory-intensive.

Stochastic Gradient Descent

Definition

SGD updates weights after seeing just one example (or a small "mini-batch").

- Pros: Faster, noise helps escape local minima
- Cons: Zigzaggy path to convergence

Backpropagation

The Chain Rule in Action

The Process

Example

Backpropagation

The Chain Rule in Action

How do we know which weight in Layer 1 caused the error in Layer 5? We use the Chain Rule of calculus.

The Process

Example

Backpropagation

The Chain Rule in Action

How do we know which weight in Layer 1 caused the error in Layer 5? We use the Chain Rule of calculus.

The Process

Example

1. Forward Pass: Data flows through network, loss is calculated

Backpropagation

The Chain Rule in Action

How do we know which weight in Layer 1 caused the error in Layer 5? We use the Chain Rule of calculus.

The Process

Example

1. Forward Pass: Data flows through network, loss is calculated
2. Backward Pass: Gradient propagates backwards layer by layer

Backpropagation

The Chain Rule in Action

How do we know which weight in Layer 1 caused the error in Layer 5? We use the Chain Rule of calculus.

The Process

Example

1. Forward Pass: Data flows through network, loss is calculated
2. Backward Pass: Gradient propagates backwards layer by layer
   - "Layer 4, you contributed X to the error. Adjust your weights."

Backpropagation

The Chain Rule in Action

How do we know which weight in Layer 1 caused the error in Layer 5? We use the Chain Rule of calculus.

The Process

Example

1. Forward Pass: Data flows through network, loss is calculated
2. Backward Pass: Gradient propagates backwards layer by layer
   - "Layer 4, you contributed X to the error. Adjust your weights."
   - "Layer 3, based on Layer 4's error, you contributed Y..."

The Vanishing Gradient Problem

Why Deep Networks Failed (Initially)

The Consequence

Example

The Vanishing Gradient Problem

Why Deep Networks Failed (Initially)

In deep networks, gradients are multiplied repeatedly during backpropagation. Sigmoid has max derivative of 0.25:

$$0.25 \times 0.25 \times 0.25 \times \cdots \approx 0$$

The Consequence

Example

The Vanishing Gradient Problem

Why Deep Networks Failed (Initially)

In deep networks, gradients are multiplied repeatedly during backpropagation. Sigmoid has max derivative of 0.25:

$$0.25 \times 0.25 \times 0.25 \times \cdots \approx 0$$

The Consequence

Example

- Gradient "vanishes" before reaching early layers

The Vanishing Gradient Problem

Why Deep Networks Failed (Initially)

In deep networks, gradients are multiplied repeatedly during backpropagation. Sigmoid has max derivative of 0.25:

$$0.25 \times 0.25 \times 0.25 \times \cdots \approx 0$$

The Consequence

Example

- Gradient "vanishes" before reaching early layers
- Early layers never learn; deep network fails

The Vanishing Gradient Problem

Why Deep Networks Failed (Initially)

In deep networks, gradients are multiplied repeatedly during backpropagation. Sigmoid has max derivative of 0.25:

$$0.25 \times 0.25 \times 0.25 \times \cdots \approx 0$$

The Consequence

Example

- Gradient "vanishes" before reaching early layers
- Early layers never learn; deep network fails
- Solution: Use ReLU (derivative = 1 for positive inputs)

The Learning Rate

The Most Critical Hyperparameter

The Goldilocks Zone

Example

Learning Rate	Behavior
Too Small	Loss decreases very slowly (0.0001)
Too Large	Loss oscillates or explodes (10.0)
Just Right	Smooth, rapid descent to minimum (0.01-0.1)

The Learning Rate

The Most Critical Hyperparameter

The Learning Rate (α) controls step size during gradient descent.

The Goldilocks Zone

Example

Learning Rate	Behavior
Too Small	Loss decreases very slowly (0.0001)
Too Large	Loss oscillates or explodes (10.0)
Just Right	Smooth, rapid descent to minimum (0.01-0.1)

The Learning Rate

The Most Critical Hyperparameter

The Learning Rate (α) controls step size during gradient descent.

The Goldilocks Zone

Example

Learning Rate	Behavior
Too Small	Loss decreases very slowly (0.0001)
Too Large	Loss oscillates or explodes (10.0)
Just Right	Smooth, rapid descent to minimum (0.01-0.1)

Weight Initialization

Starting Right

Strategies

Example

Weight Initialization

Starting Right

Initialization	Problem
All Zeros	Symmetry: every neuron learns same thing
Too Large	Gradients explode
Too Small	Gradients vanish

Strategies

Example

Weight Initialization

Starting Right

Initialization	Problem
All Zeros	Symmetry: every neuron learns same thing
Too Large	Gradients explode
Too Small	Gradients vanish

Strategies

Example

- He Initialization: Best for ReLU networks

Weight Initialization

Starting Right

Initialization	Problem
All Zeros	Symmetry: every neuron learns same thing
Too Large	Gradients explode
Too Small	Gradients vanish

Strategies

Example

- He Initialization: Best for ReLU networks
- Xavier (Glorot): Best for Sigmoid/Tanh networks

Weight Initialization

Starting Right

Initialization	Problem
All Zeros	Symmetry: every neuron learns same thing
Too Large	Gradients explode
Too Small	Gradients vanish

Strategies

Example

- He Initialization: Best for ReLU networks
- Xavier (Glorot): Best for Sigmoid/Tanh networks
- Both keep variance of activations constant across layers

Advanced Optimizers

Smarter Descent

Advanced Optimizers

Smarter Descent

Standard SGD is like walking down a mountain blindfolded. Advanced optimizers add intelligence.

Advanced Optimizers

Smarter Descent

Standard SGD is like walking down a mountain blindfolded. Advanced optimizers add intelligence.

Optimizer	Mechanism	Analogy
SGD + Momentum	Accumulates past gradients	Heavy ball rolling downhill
RMSProp	Adapts learning rate per parameter	Slows on steep, speeds on flat
Adam	Momentum + RMSProp combined	Smart ball, adapts speed & direction

Advanced Optimizers

Smarter Descent

Standard SGD is like walking down a mountain blindfolded. Advanced optimizers add intelligence.

Optimizer	Mechanism	Analogy
SGD + Momentum	Accumulates past gradients	Heavy ball rolling downhill
RMSProp	Adapts learning rate per parameter	Slows on steep, speeds on flat
Adam	Momentum + RMSProp combined	Smart ball, adapts speed & direction

Adam is the default choice for most tasks.

Regularization: Dropout

Fighting Overfitting

Definition

Why It Works

Example

Regularization: Dropout

Fighting Overfitting

Definition

Dropout randomly "kills" (sets to zero) a percentage of neurons during training.

Why It Works

Example

Regularization: Dropout

Fighting Overfitting

Definition

Dropout randomly "kills" (sets to zero) a percentage of neurons during training.

Why It Works

Example

- Prevents neurons from co-adapting

Regularization: Dropout

Fighting Overfitting

Definition

Dropout randomly "kills" (sets to zero) a percentage of neurons during training.

Why It Works

Example

- Prevents neurons from co-adapting
- Forces network to be redundant and robust

Regularization: Dropout

Fighting Overfitting

Definition

Dropout randomly "kills" (sets to zero) a percentage of neurons during training.

Why It Works

Example

- Prevents neurons from co-adapting
- Forces network to be redundant and robust
- Each training pass uses a different "thinned" network

Regularization: Dropout

Fighting Overfitting

Definition

Dropout randomly "kills" (sets to zero) a percentage of neurons during training.

Why It Works

Example

- Prevents neurons from co-adapting
- Forces network to be redundant and robust
- Each training pass uses a different "thinned" network
- At test time, all neurons are used (with scaled weights)

Regularization: Batch Normalization

Stabilizing Training

Definition

Benefits

Example

Regularization: Batch Normalization

Stabilizing Training

Definition

Batch Normalization normalizes inputs of each layer to have mean 0 and variance 1 during training.

Benefits

Example

Regularization: Batch Normalization

Stabilizing Training

Definition

Batch Normalization normalizes inputs of each layer to have mean 0 and variance 1 during training.

Benefits

Example

- Stabilizes training (allows higher learning rates)

Regularization: Batch Normalization

Stabilizing Training

Definition

Batch Normalization normalizes inputs of each layer to have mean 0 and variance 1 during training.

Benefits

Example

- Stabilizes training (allows higher learning rates)
- Reduces sensitivity to initialization

Regularization: Batch Normalization

Stabilizing Training

Definition

Batch Normalization normalizes inputs of each layer to have mean 0 and variance 1 during training.

Benefits

Example

- Stabilizes training (allows higher learning rates)
- Reduces sensitivity to initialization
- Acts as a form of regularization

Regularization: Batch Normalization

Stabilizing Training

Definition

Batch Normalization normalizes inputs of each layer to have mean 0 and variance 1 during training.

Benefits

Example

- Stabilizes training (allows higher learning rates)
- Reduces sensitivity to initialization
- Acts as a form of regularization
- Enables training of much deeper networks

Early Stopping

The Simplest Regularization

How It Works

Example

Early Stopping

The Simplest Regularization

Monitor the Validation Loss. If it stops improving (or starts getting worse), stop training immediately.

How It Works

Example

Early Stopping

The Simplest Regularization

Monitor the Validation Loss. If it stops improving (or starts getting worse), stop training immediately.

How It Works

Example

- Training loss keeps decreasing (memorization)

Early Stopping

The Simplest Regularization

Monitor the Validation Loss. If it stops improving (or starts getting worse), stop training immediately.

How It Works

Example

- Training loss keeps decreasing (memorization)
- Validation loss starts increasing (overfitting)

Early Stopping

The Simplest Regularization

Monitor the Validation Loss. If it stops improving (or starts getting worse), stop training immediately.

How It Works

Example

- Training loss keeps decreasing (memorization)
- Validation loss starts increasing (overfitting)
- Early stopping saves the model at the sweet spot

Early Stopping

The Simplest Regularization

Monitor the Validation Loss. If it stops improving (or starts getting worse), stop training immediately.

How It Works

Example

- Training loss keeps decreasing (memorization)
- Validation loss starts increasing (overfitting)
- Early stopping saves the model at the sweet spot
- No hyperparameter tuning needed (just patience parameter)

Why PyTorch?

Beyond Scikit-learn

PyTorch Advantages

Example

- Readable, "Pythonic" style
- Research and education standard
- Dynamic computation graphs (easy debugging)

Why PyTorch?

Beyond Scikit-learn

Scikit-learn is great for standard ML, but Deep Learning needs:

PyTorch Advantages

Example

- Readable, "Pythonic" style
- Research and education standard
- Dynamic computation graphs (easy debugging)

Why PyTorch?

Beyond Scikit-learn

Scikit-learn is great for standard ML, but Deep Learning needs:

1. GPU Acceleration: Matrix multiplication is slow on CPUs

PyTorch Advantages

Example

- Readable, "Pythonic" style
- Research and education standard
- Dynamic computation graphs (easy debugging)

Why PyTorch?

Beyond Scikit-learn

Scikit-learn is great for standard ML, but Deep Learning needs:

1. GPU Acceleration: Matrix multiplication is slow on CPUs
2. Automatic Differentiation: Backpropagation is hard to code by hand

PyTorch Advantages

Example

- Readable, "Pythonic" style
- Research and education standard
- Dynamic computation graphs (easy debugging)

Why PyTorch?

Beyond Scikit-learn

Scikit-learn is great for standard ML, but Deep Learning needs:

1. GPU Acceleration: Matrix multiplication is slow on CPUs
2. Automatic Differentiation: Backpropagation is hard to code by hand

PyTorch Advantages

Example

- Readable, "Pythonic" style
- Research and education standard
- Dynamic computation graphs (easy debugging)

Why PyTorch?

Beyond Scikit-learn

Scikit-learn is great for standard ML, but Deep Learning needs:

1. GPU Acceleration: Matrix multiplication is slow on CPUs
2. Automatic Differentiation: Backpropagation is hard to code by hand

PyTorch Advantages

Example

- Readable, "Pythonic" style
- Research and education standard
- Dynamic computation graphs (easy debugging)

Tensors: The Foundation

NumPy Arrays on Steroids

Definition

Key Operations

Example

Tensors: The Foundation

NumPy Arrays on Steroids

Definition

A Tensor is like a NumPy array that:

- Can live on a GPU
- Remembers how it was created (for backprop)

Key Operations

Example

Tensors: The Foundation

NumPy Arrays on Steroids

Definition

A Tensor is like a NumPy array that:

- Can live on a GPU
- Remembers how it was created (for backprop)

Key Operations

Example

- `torch.tensor([1, 2, 3])` — Create from list

Tensors: The Foundation

NumPy Arrays on Steroids

Definition

A Tensor is like a NumPy array that:

- Can live on a GPU
- Remembers how it was created (for backprop)

Key Operations

Example

- `torch.tensor([1, 2, 3])` — Create from list
- `torch.ones(2, 3)` — Shape (2, 3) of ones

Tensors: The Foundation

NumPy Arrays on Steroids

Definition

A Tensor is like a NumPy array that:

- Can live on a GPU
- Remembers how it was created (for backprop)

Key Operations

Example

- `torch.tensor([1, 2, 3])` — Create from list
- `torch.ones(2, 3)` — Shape (2, 3) of ones
- `x.to("cuda")` — Move to GPU

Tensors: The Foundation

NumPy Arrays on Steroids

Definition

A Tensor is like a NumPy array that:

- Can live on a GPU
- Remembers how it was created (for backprop)

Key Operations

Example

- `torch.tensor([1, 2, 3])` — Create from list
- `torch.ones(2, 3)` — Shape (2, 3) of ones
- `x.to("cuda")` — Move to GPU
- Math: `x * 2 + 5` — Same as NumPy

Defining a Network (`nn.Module`)

The Blueprint

Two Required Methods

Example

Defining a Network (`nn.Module`)

The Blueprint

In PyTorch, a network is a Python class inheriting from `nn.Module`:

Two Required Methods

Example

Defining a Network (`nn.Module`)

The Blueprint

In PyTorch, a network is a Python class inheriting from `nn.Module`:

Two Required Methods

Example

1. `__init__`: Define the layers

Defining a Network (`nn.Module`)

The Blueprint

In PyTorch, a network is a Python class inheriting from `nn.Module`:

Two Required Methods

Example

1. `__init__`: Define the layers
   - `self.layer1 = nn.Linear(input_dim, hidden_dim)`

Defining a Network (`nn.Module`)

The Blueprint

In PyTorch, a network is a Python class inheriting from `nn.Module`:

Two Required Methods

Example

1. `__init__`: Define the layers
   - `self.layer1 = nn.Linear(input_dim, hidden_dim)`
   - `self.activation = nn.ReLU()`

Defining a Network (`nn.Module`)

The Blueprint

In PyTorch, a network is a Python class inheriting from `nn.Module`:

Two Required Methods

Example

1. `__init__`: Define the layers
   - `self.layer1 = nn.Linear(input_dim, hidden_dim)`
   - `self.activation = nn.ReLU()`
2. `forward`: Define data flow

Defining a Network (`nn.Module`)

The Blueprint

In PyTorch, a network is a Python class inheriting from `nn.Module`:

Two Required Methods

Example

1. `__init__`: Define the layers
   - `self.layer1 = nn.Linear(input_dim, hidden_dim)`
   - `self.activation = nn.ReLU()`
2. `forward`: Define data flow
   - `x = self.layer1(x)`

Defining a Network (`nn.Module`)

The Blueprint

In PyTorch, a network is a Python class inheriting from `nn.Module`:

Two Required Methods

Example

1. `__init__`: Define the layers
   - `self.layer1 = nn.Linear(input_dim, hidden_dim)`
   - `self.activation = nn.ReLU()`
2. `forward`: Define data flow
   - `x = self.layer1(x)`
   - `x = self.activation(x)`

Defining a Network (`nn.Module`)

The Blueprint

In PyTorch, a network is a Python class inheriting from `nn.Module`:

Two Required Methods

Example

1. `__init__`: Define the layers
   - `self.layer1 = nn.Linear(input_dim, hidden_dim)`
   - `self.activation = nn.ReLU()`
2. `forward`: Define data flow
   - `x = self.layer1(x)`
   - `x = self.activation(x)`
   - `return x`

The Training Loop

The 5-Step Boilerplate

The Training Loop

The 5-Step Boilerplate

Unlike Scikit-learn's `model.fit()`, PyTorch requires manual control:

The Training Loop

The 5-Step Boilerplate

Unlike Scikit-learn's `model.fit()`, PyTorch requires manual control:

1. Forward Pass: `y_pred = model(X)`

The Training Loop

The 5-Step Boilerplate

Unlike Scikit-learn's `model.fit()`, PyTorch requires manual control:

1. Forward Pass: `y_pred = model(X)`
2. Compute Loss: `loss = criterion(y_pred, y)`

The Training Loop

The 5-Step Boilerplate

Unlike Scikit-learn's `model.fit()`, PyTorch requires manual control:

1. Forward Pass: `y_pred = model(X)`
2. Compute Loss: `loss = criterion(y_pred, y)`
3. Zero Gradients: `optimizer.zero_grad()`

The Training Loop

The 5-Step Boilerplate

Unlike Scikit-learn's `model.fit()`, PyTorch requires manual control:

1. Forward Pass: `y_pred = model(X)`
2. Compute Loss: `loss = criterion(y_pred, y)`
3. Zero Gradients: `optimizer.zero_grad()`
4. Backward Pass: `loss.backward()`

The Training Loop

The 5-Step Boilerplate

Unlike Scikit-learn's `model.fit()`, PyTorch requires manual control:

1. Forward Pass: `y_pred = model(X)`
2. Compute Loss: `loss = criterion(y_pred, y)`
3. Zero Gradients: `optimizer.zero_grad()`
4. Backward Pass: `loss.backward()`
5. Update Weights: `optimizer.step()`

The Training Loop

The 5-Step Boilerplate

Unlike Scikit-learn's `model.fit()`, PyTorch requires manual control:

1. Forward Pass: `y_pred = model(X)`
2. Compute Loss: `loss = criterion(y_pred, y)`
3. Zero Gradients: `optimizer.zero_grad()`
4. Backward Pass: `loss.backward()`
5. Update Weights: `optimizer.step()`

The Training Loop

The 5-Step Boilerplate

Unlike Scikit-learn's `model.fit()`, PyTorch requires manual control:

1. Forward Pass: `y_pred = model(X)`
2. Compute Loss: `loss = criterion(y_pred, y)`
3. Zero Gradients: `optimizer.zero_grad()`
4. Backward Pass: `loss.backward()`
5. Update Weights: `optimizer.step()`

Wrap in a `for epoch in range(epochs):` loop.

Loss Functions and Optimizers

PyTorch Components

Loss Functions

Example

Task	Loss Class
Binary Classification	`nn.BCELoss()`
Multiclass	`nn.CrossEntropyLoss()`
Regression	`nn.MSELoss()`

Optimizers

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

Loss Functions and Optimizers

PyTorch Components

Loss Functions

Example

Task	Loss Class
Binary Classification	`nn.BCELoss()`
Multiclass	`nn.CrossEntropyLoss()`
Regression	`nn.MSELoss()`

Optimizers

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

Loss Functions and Optimizers

PyTorch Components

Loss Functions

Example

Task	Loss Class
Binary Classification	`nn.BCELoss()`
Multiclass	`nn.CrossEntropyLoss()`
Regression	`nn.MSELoss()`

Optimizers

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

Evaluation Mode

No Gradients Needed

Evaluation Mode

No Gradients Needed

When evaluating, we don't need to calculate gradients. Use `torch.no_grad()` to save memory:

Evaluation Mode

No Gradients Needed

When evaluating, we don't need to calculate gradients. Use `torch.no_grad()` to save memory:

```
with torch.no_grad():
    test_preds = model(X_test)
    predicted = (test_preds > 0.5).float()
    accuracy = (predicted == y_test).float().mean()
```

Key Takeaways

What We Learned

Key Takeaways

What We Learned

1. ANNs: Layers of neurons with weights and activation functions

Key Takeaways

What We Learned

1. ANNs: Layers of neurons with weights and activation functions
2. Perceptron: Single neuron, limited to linear problems

Key Takeaways

What We Learned

1. ANNs: Layers of neurons with weights and activation functions
2. Perceptron: Single neuron, limited to linear problems
3. MLP: Stacked neurons solve non-linear problems (XOR)

Key Takeaways

What We Learned

1. ANNs: Layers of neurons with weights and activation functions
2. Perceptron: Single neuron, limited to linear problems
3. MLP: Stacked neurons solve non-linear problems (XOR)
4. Training: Forward pass $\rightarrow$ Loss $\rightarrow$ Backprop $\rightarrow$ Update

Key Takeaways

What We Learned

1. ANNs: Layers of neurons with weights and activation functions
2. Perceptron: Single neuron, limited to linear problems
3. MLP: Stacked neurons solve non-linear problems (XOR)
4. Training: Forward pass $\rightarrow$ Loss $\rightarrow$ Backprop $\rightarrow$ Update
5. Vanishing Gradient: Sigmoid fails deep; ReLU fixes it

Key Takeaways

What We Learned

1. ANNs: Layers of neurons with weights and activation functions
2. Perceptron: Single neuron, limited to linear problems
3. MLP: Stacked neurons solve non-linear problems (XOR)
4. Training: Forward pass $\rightarrow$ Loss $\rightarrow$ Backprop $\rightarrow$ Update
5. Vanishing Gradient: Sigmoid fails deep; ReLU fixes it
6. Optimization: Adam is the default; Early Stopping prevents overfitting

Key Takeaways

What We Learned

1. ANNs: Layers of neurons with weights and activation functions
2. Perceptron: Single neuron, limited to linear problems
3. MLP: Stacked neurons solve non-linear problems (XOR)
4. Training: Forward pass $\rightarrow$ Loss $\rightarrow$ Backprop $\rightarrow$ Update
5. Vanishing Gradient: Sigmoid fails deep; ReLU fixes it
6. Optimization: Adam is the default; Early Stopping prevents overfitting
7. PyTorch: Tensors + nn.Module + Training Loop

Next Section Preview

Neural Networks II: Deep Learning & Transformers

Next Section Preview

Neural Networks II: Deep Learning & Transformers

- Convolutional Neural Networks (CNNs): Image processing with filters

Next Section Preview

Neural Networks II: Deep Learning & Transformers

- Convolutional Neural Networks (CNNs): Image processing with filters
- From RNNs to Transformers: Sequence modeling revolution

Next Section Preview

Neural Networks II: Deep Learning & Transformers

- Convolutional Neural Networks (CNNs): Image processing with filters
- From RNNs to Transformers: Sequence modeling revolution
- Large Language Models: GPT, BERT, and modern NLP

Next Section Preview

Neural Networks II: Deep Learning & Transformers

- Convolutional Neural Networks (CNNs): Image processing with filters
- From RNNs to Transformers: Sequence modeling revolution
- Large Language Models: GPT, BERT, and modern NLP
- Generative AI: Text generation, RAG, Diffusion models